1 Evaluation Methodology

This chapter details the methodology used to evaluate the Agentic Project Management (APM) framework. To assess the framework's effectiveness against current industry standards, we conducted a comparative case study involving human participants. The study compares APM, a multi-agent framework that distributes work across specialized agents, against GitHub Spec-kit, a single-agent toolkit that guides one AI agent through a structured specification-driven workflow. Both frameworks are described in Section 1.2.

The evaluation measures three distinct dimensions:

- Economic Efficiency *[Cost framing forthcoming.]*
- Process Efficiency (human cognitive load and interaction burden)
- Functional Efficacy (ability to complete non-trivial tasks)

[Overview of the Copilot-based comparative study — forthcoming.]

1.1 Research Objectives

The primary objective of this research is to determine whether the structural overhead introduced by APM (i.e., the Planning and Implementation phases) yields a tangible return on investment compared to a single-agent workflow. We define the following Research Questions (RQs):

- **RQ1 (Economic Efficiency):** *[Forthcoming. To be reframed around request consumption and resource efficiency rather than raw token cost.]*
- **RQ2 (Process Efficiency):**
  - Does the structured coordination of APM reduce the amount of manual effort the user must invest during execution, particularly interventions, corrections, and clarifications?
  - If a difference in user effort is observed, is it significant enough to justify the structural overhead APM introduces?
  - Given that the User remains in the loop as a message carrier, can the agents coordinate effectively enough that the User's role stays limited to shuttling messages rather than correcting or redirecting agent behavior?
- **RQ3 (Functional Efficacy):**
  - Can APM successfully deliver complex, non-trivial features in scenarios where single-agent workflows are prone to failure due to context exhaustion, loss of coherence across long sessions, or inability to coordinate work across multiple concerns?
  - How does the performance of APM compare to the performance of a single-agent workflow?

1.2 Frameworks Under Evaluation

The two frameworks are both open-source and spec-driven, running on the same AI IDE platform. Both structure work through explicit specification and planning artifacts before implementation begins. They differ in how execution is organized: Spec-kit operates within a single agent conversation, while APM distributes work across multiple specialized agents.

1.2.1 GitHub Spec-kit

Spec-kit[1] is a specification-driven development toolkit developed by GitHub. It guides a single AI agent through a structured workflow using three sequential commands:

1. `/speckit.specify`: transforms a feature description into a structured specification containing requirements, user stories, and acceptance criteria.
2. `/speckit.plan`: converts the specification into a technical implementation plan with architecture decisions, data models, API contracts, and supporting documents.
3. `/speckit.tasks`: derives an executable task list (`tasks.md`) from the plan, with parallelism annotations and sequencing constraints.

Additional commands support iterative refinement, plan validation, and execution. A constitution mechanism allows users to establish architectural rules that the agent enforces throughout the workflow.

All planning and implementation happens within one continuous conversation. The agent acts as both planner and executor, and the full specification, plan, and task context accumulates in a single context window. Spec-kit produces a fixed set of artifact types (specification, plan, data model, contracts, research, tasks) regardless of project type; the templates are predefined and every project receives the same artifact layout.

1.2.2 APM in Comparison

Where Spec-kit runs the entire workflow in a single conversation, APM (described in detail in the preceding chapters) separates planning, coordination, and execution across three agent roles, each operating in its own conversation with deliberately scoped context. Agents communicate through a file-based Message Bus rather than sharing a context window, planning documents adapt their structure to the project rather than matching to a predetermined artifact layout, and the user acts as a message carrier between conversations. When an agent's context fills, an explicit Handoff mechanism preserves working knowledge across context boundaries.

The evaluation tests whether this multi-agent overhead produces measurably better outcomes than Spec-kit's single-agent approach. The task is designed to be complex enough to challenge a single context window, making it a meaningful test of whether distributing work across specialized agents with scoped context yields tangible benefits in practice.

[1] Documentation available at: https://github.github.com/spec-kit/

1.3 Testbed and Task Design

To evaluate the frameworks under realistic constraints, we utilized an existing open-source project, the `eclass-mcp-server`, a Python-based Model Context Protocol (MCP) server designed to interface with the `openeclass` platform. Openeclass is an online platform for Greek universities that allows students to manage their courses and assignments.

1.3.1 Task Description

The task requires participants to implement two new announcement tools in the Python MCP server and to create a complete native C replica of the entire server.[2] The two tools are:

- `get_course_announcements`: retrieves announcements for a specific course.
- `get_general_announcements`: aggregates system-wide and per-course announcements.

The C implementation adds significant complexity: participants must manage code translation, memory safety, and protocol details across two very different languages. This combination of feature implementation and full-language replication is calibrated to stress-test the limits of agent coordination and expose any differences between the two frameworks, especially when context management becomes a bottleneck.

Key technical challenges:

1. Dynamic Data Source: the required data for implementing the new features is not available in static HTML for parsing. Agents must research the legacy codebase to discover a specific AJAX endpoint and its behavior.
2. Nested Parsing: the server returns JSON where the payload is an array of raw HTML strings, requiring a second layer of parsing to extract the data.
3. Client-Side Filtering: the legacy API does not support date filtering. The agent must implement robust client-side logic to parse and filter unique locale-formatted dates.
4. C Implementation: agents must build a native C MCP server replica, manually implementing JSON-RPC 2.0 with standard C string manipulation (no external JSON libraries such as cJSON/Jansson). Use of `libcurl` for HTTP/HTTPS and `libxml2` for HTML parsing was required.

Deliverables:

1. Core server logic in modular Python files (existing and new functionality).
2. A native C implementation of the MCP server with equivalent functionality.
3. A working build system (Makefile) for the C replica.

[2] Task specification and grading infrastructure available at:
https://github.com/sdi2200262/bsc-apm-case-study-task

1.3.2 Grading

Submissions are evaluated using an automated black-box test suite with comprehensive validation, grading, metric extraction, and dataset aggregation capabilities (see the evaluation appendix for detailed infrastructure documentation).[3]

The test suite awards 100 points total: 12 points for baseline functionality (existing Python tools), 36 points for the two new Python features (18 points each), and 52 points for the complete C replica implementation. A submission with no patch applied receives the baseline score of 12/100 points.

All tests run against fixed baseline commits to ensure reproducibility: `eclass-mcp-server` at commit `8b4e39c` and `openeclass` at commit `e8b3329`.

1.3.3 Repository Architecture and Data Management

The case study employs a two-repository system separating public resources from sensitive participant data. The task repository (`bsc-apm-case-study-task`) is openly accessible and includes task specifications, grading infrastructure, and participant guides. The dataset repository (`bsc-apm-case-study-data`) contains all submitted participant data: code patches, sanitized chat transcripts (see Section 1.3.4), and anonymized session metadata.

Access to the dataset repository requires explicit authorization and compliance with data usage agreements, enabling peer review and reproducibility while maintaining participant privacy.

1.3.4 Sanitization and PII Handling

Chat exports contain sensitive information (unstructured user-input text, file paths, command histories, timestamps). To protect participant privacy while enabling reproducibility, all exports are sanitized using reversible encryption before storage.

Each chat is assigned a unique identifier via SHA256 hashing of concatenated user messages for anonymous cataloging. Essential metadata (chat ID, model IDs, request count) is preserved in plaintext to enable dataset organization without decryption, while conversation content is encrypted using an HMAC-SHA256 XOR cipher. Detailed technical specifications are provided in the evaluation appendix.

[3] Grading infrastructure available at: https://github.com/sdi2200262/bsc-apm-case-study-task/tree/main/grader

1.4 Study Design

The study is conducted on a single AI IDE platform, VS Code with GitHub Copilot, and compares the two frameworks across a homogeneous cohort of participants. It is designed as a within-subjects experiment, so that each participant contributes data points for both frameworks and per-participant variance in skill, prior experience, and working style is absorbed into each pairwise comparison rather than confounding it.

1.4.1 Experimental Design

Each participant completes the case study task twice: once with APM, once with Spec-kit. The order of the two frameworks is counterbalanced across participants, so that approximately half of the cohort encounters APM first and the other half encounters Spec-kit first. This counterbalancing lets us attribute any observed first-to-second session improvement to task familiarity rather than to the framework itself, and is analyzed explicitly in Section 2 under Learning Effects.

The two sessions are isolated from one another using Git, so no code, state, or context from the first session can influence the second. The participation workflow including session isolation and patch submission are documented in the evaluation appendix.

1.4.2 Data Source

The primary data source for the study is GitHub Copilot chat exports, captured in JSON form from the VS Code Command Palette at the end of each session, alongside each participant's patch submission. Copilot's export format provides structured records of every conversation, including model identifiers, message timestamps, tool invocations, and per-message speaker roles. This granularity enables programmatic extraction of both request-level and interaction-level metrics directly from the exports, without manual annotation.

APM's multi-agent workflow spans multiple Copilot conversations within a single session (one for the Planner, one for the Manager, and multiple for the Workers), while Spec-kit operates within a single conversation. To keep the two frameworks comparable, all metrics are computed by aggregating every conversation file belonging to a participant submission into a unified session-level view, so that the total interaction cost of the session is captured regardless of how the work was distributed across conversations.

1.4.3 Participant Pool

Participants are drawn from a cohort of University students and working professionals with active access to GitHub Copilot, either through the GitHub Education Pack or through a standard premium subscription.

1.5 Metrics and Data Collection

The metrics collected for this study are organized into five categories: session metadata, the three dimensions introduced above (economic efficiency, process efficiency, and functional efficacy), and the functional grading rubric. Concrete definitions for most categories depend on the cost framing, which drives several metric-level design choices; these are marked as forthcoming below.

1.5.1 Session Metadata

[Forthcoming. Will cover the distinct model identifiers used across a session, the per-conversation and project-level timestamps, and the session count for each APM submission.]

1.5.2 Cost Metrics

[Forthcoming. Will operationalise Copilot request consumption as a proxy for enterprise budget impact, with request-per-functional-point as the primary efficiency ratio.]

1.5.3 Reliability Metrics

[Forthcoming. Will cover platform-agnostic measures of system competence — tool success rate, tool retry rate, and user edit rate on terminal commands — as indicators of how autonomously each framework executes work.]

1.5.4 Process Metrics

[Forthcoming. Will cover behavior distribution (ratios of short, protocol, and standard user inputs), behavior magnitude (average characters per input bucket), and session duration, as indicators of the human coordination burden each framework imposes.]

1.5.5 Functional Metrics

The functional dimension is evaluated against the 100-point grading rubric introduced in Section 1.3.2. Final code artifacts (`.patch` files) are scored across three dimensions:

- Baseline Regression: ensures that existing functionality (12 points) remains intact after the patch is applied, isolating the framework's impact on code stability.
- Functional Correctness: end-to-end validation of the new Python features (36 points) and the complete C replica (52 points) against the specification.
- Robustness: edge-case handling including authentication failures, invalid inputs, empty responses, and memory safety (Valgrind validation for the C implementation).

2 Results

2.1 Functional Performance Analysis

Primary metrics: total score distribution, completion rates (100/100, 49–99, 12–48, baseline-only), baseline regression rates, and feature implementation success rates for Python and C.

Category analysis: success rates per test category (build, protocol, login, logout, courses, auth-status, announcements) for both Python and C implementations, identifying framework-specific failure patterns.

Comparative analysis: t-tests comparing APM vs. Spec-kit for total score, category scores, and completion rates, with effect sizes (Cohen's d) to assess practical significance.

2.2 Economic and Process Efficiency Analysis

[Forthcoming. Unified analysis of the economic and process metrics introduced in Section 1.5, correlated with the functional outcomes.]

2.3 Learning Effects and Confounding

Participants complete the same task twice (Session 1 and Session 2) using different frameworks. This creates a learning effect where task familiarity from S1 influences S2 performance, potentially confounding framework comparisons.

Analysis strategy:

- Learning Effect Matrix: calculate mean improvement from S1 to S2 for both framework orderings across all metrics (functional scores, cost, process efficiency).
- Order Effect: compare APM-first vs. Spec-kit-first directions to test whether order of framework exposure affects learning.
- Baseline Learning: overall mean improvement across both directions (pure task learning effect regardless of framework).

Key questions: does task learning confound framework comparisons? Is there a preferred first framework that maximizes S2 improvement? How should learning effects be controlled for when comparing frameworks?

3 Discussion

3.1 Cost-Benefit Analysis: Context Overhead vs. Failure Prevention

[Forthcoming. Synthesises the cost metrics with functional outcomes to assess whether APM's structural overhead is justified by improved reliability.]

3.2 Process Efficiency: Protocol Overhead vs. Intervention Reduction

[Forthcoming. Synthesises the process metrics to assess whether APM's coordination overhead is offset by reduced intervention burden on the User.]

3.3 Functional Efficacy: Reliability and Failure Patterns

[Forthcoming. Synthesises functional scores, completion rates, and failure patterns to characterise each framework's reliability.]

3.4 Learning Effects: Confounding and Framework Transferability

[Forthcoming. Interprets the learning-effect matrix from Section 2 and discusses how first-to-second session improvement affects the framework comparison.]

3.5 Threats to Validity

[Forthcoming. Enumerates the internal, external, and construct validity threats relevant to the study and the mitigations applied.]